

Experimental Evaluation and Optimization of Real-Time Scheduling on STM32

Vladimir Toporkov
Computer Science and Engineering
Innopolis University
Innopolis, Russia
v.toporkov@innopolis.university

Artem Burmyakov
Institute of Information Security
Innopolis University
Innopolis, Russia
a.burmyakov@innopolis.ru

I. INTRODUCTION

A. Contribution

We propose the following contribution:

- Experimental evaluation and comparison of Super Loop and Earliest Deadline First on STM32.

II. BACKGROUND ON STM32

A. Hardware

The experimental platform consists of a single-core STM32 microcontroller and external sensors connected through its hardware peripherals, as shown in Fig. 1.

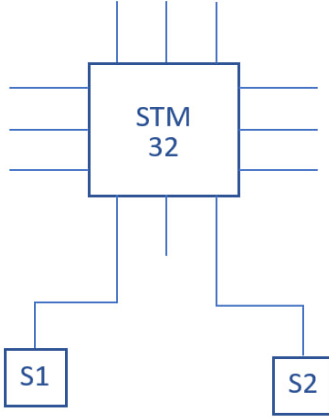


Fig. 1. STM32 hardware platform and connected sensors.

The hardware components used in the experiments are summarized in Table I.

TABLE I
EXPERIMENTAL HARDWARE PLATFORM.

Component	Details	Purpose
STM32F767	Cortex-M7, TODO MHz	Scheduling and task execution
Sensor 1	TODO	TODO
Sensor 2	TODO	TODO
Sensor 3	TODO	TODO
Sensor 4	TODO	TODO

B. Super Loop Scheduler

Let us first consider an example of a Super Loop schedule for two periodic tasks with implicit deadlines ($D_i = T_i$).

TABLE II
TASK SET USED IN THE SUPER LOOP EXAMPLE.

Task	C_i	T_i	D_i
τ_1	20	50	50
τ_2	10	20	20

Here, C_i , T_i , and D_i denote execution time, period, and relative deadline respectively.

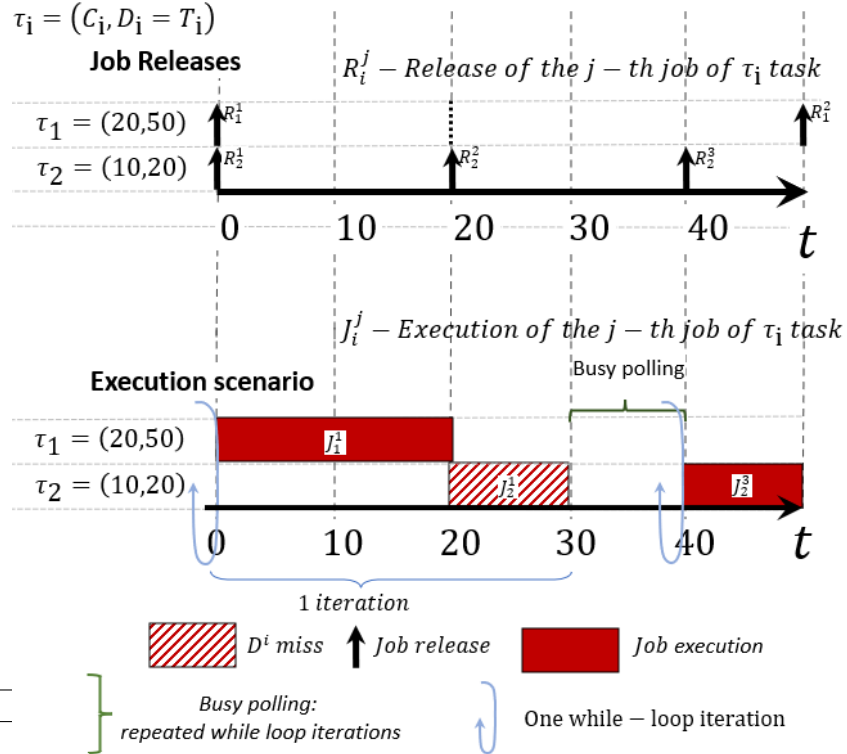


Fig. 2. Super Loop execution scenario for the task set in Table II.

Super Loop checks tasks sequentially in a fixed order, while

each selected job executes to completion before the next task is checked.

The corresponding scheduling state is shown in Table III.

A deadline miss is detected when $t^{cur} > R_i^k + D_i$, while a release is skipped when $t^{cur} > t_i^{next}$.

TABLE III
SUPER LOOP SCHEDULING STATE FOR FIG. 2.

t^{cur}	t_1^{next}	t_2^{next}	Deadline miss check	Release skip check
0	0	0	—	—
20	50	0	$20 < R_1^1 + 50$	$20 < R_2^2$
30	50	40	$30 > R_2^1 + 20$	$30 > R_2^2$
40	50	40	—	—
50	50	60	$50 < R_2^3 + 20$	$50 < R_2^4$

In Fig. 2, J_1^1 executes from $t = 0$ to $t = 20$, delaying J_2^1 beyond its deadline. At $t = 20$, τ_2 reaches its next scheduled release time while J_2^1 is still pending; therefore, this release is skipped and no J_2^2 is created.

III. BACKGROUND ON EDF REAL-TIME SCHEDULING

Super Loop is deadline-unaware: jobs are selected according to a fixed loop order rather than their deadlines. As shown in Fig. 2, this may delay jobs beyond their deadlines and cause skipped releases.

TABLE IV
SUPER LOOP AND EDF SCHEDULING PRINCIPLES.

Property	Super Loop	EDF
Job selection	Fixed loop order	Earliest deadline
Deadline-aware	No	Yes
Preemption	No	Yes

EDF dynamically selects the ready job with the earliest absolute deadline. For a job J_i^k released at R_i^k , its absolute deadline is

$$d_i^k = R_i^k + D_i$$

If a newly released job has an earlier deadline than the running job, the running job is preempted.

For the same task set as in Section II-B, the resulting EDF schedule is shown in Fig. ?? . Observe that, unlike Super Loop, EDF:

- selects the ready job with the earliest absolute deadline;
- preempts the running job when a job with an earlier deadline is released;
- avoids the deadline miss observed under Super Loop for the considered task set.

IV. SCHEDULERS IMPLEMENTATION FOR STM32

ToDo (code?)

V. PERFORMANCE EVALUATION

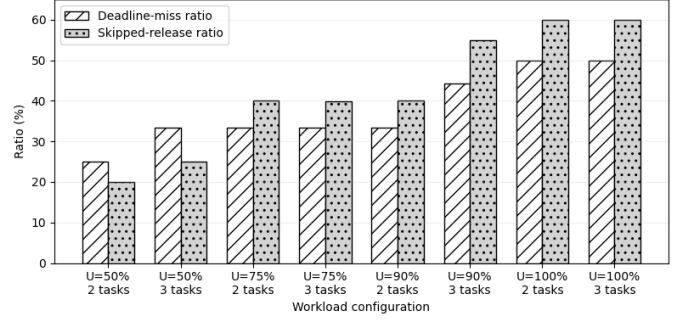


Fig. 3. Super Loop deadline-miss and skipped-release ratios for a specific task under different CPU utilization (U) levels and task-set sizes.